STRINGS

1. Remove Outermost Parentheses

Problem Statement

A **valid parentheses string** is defined as:

- `""` (empty string) is valid
- If `A` is valid, then `"(" + A + ")"` is valid
- If `A` and `B` are valid, then `A + B` is valid

A **primitive valid parentheses string** is a non-empty valid string that **cannot be split** into two or more non-empty valid parentheses strings.

Given a valid parentheses string `s`, remove the **outermost parentheses** from **every primitive** in `s` and return the resulting string.

Example 1

```
Input:
s = "((()))"

Output:
"(())"

Explanation:
Primitive = "((()))"
After removing outermost parentheses → "(())"
```

Example 2

```
Input:
s = "()(()())(())"

Output:
"(()())()"
```

```
Explanation:
Primitives:
"()" , "(()())" , "(())"

After removing outermost parentheses:
"" + "()()" + "()"

Final result: "(()())()"
```

Approach

Core Idea

We track the **depth (level)** of parentheses while traversing the string.

- The **outermost parentheses** of a primitive are:
    - The `'('` when `level == 0`
    - The `')'` when `level == 1`

So:

- We **skip** these outermost parentheses
- We **keep** all others

Steps

1. Initialize:
    - `level = 0`
    - `result = ""`

2. Traverse the string character by character:
    - If `'('`:
        - If `level > 0`, add `'('` to result
        - Increment `level`
    - If `')'`:
        - Decrement `level`
        - If `level > 0`, add `')'` to result
3. Return `result`

Why This Works

- Each primitive starts when `level` goes from `0 → 1`
- Its outermost parentheses are exactly at:
    - first `'('` (level was 0)
    - last `')'` (level becomes 0)
- By checking `level`, we automatically ignore only those outer ones

---

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
```

```
    string removeOuterParentheses(string s) {
        string result = "";
        int level = 0;

        for (char ch : s) {
            if (ch == '(') {
                if (level > 0)
                    result += ch;
                level++;
            }
            else { // ch == ')'
                level--;
                if (level > 0)
                    result += ch;
            }
        }
        return result;
    }
};

int main() {
    string s = "(()(()))(())";
    Solution sol;
    cout << sol.removeOuterParentheses(s) << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n)$ — single traversal of the string
- **Space Complexity:** $O(1)$ — excluding the output string

2. Reverse Words in a String

Problem Statement

You are given a string s consisting of:

- Uppercase and lowercase letters
- Digits
- Spaces (`' '`)

A **word** is defined as a sequence of non-space characters.
Words are separated by **one or more spaces**.

You need to:

- Reverse the **order of words**
- Ensure **exactly one space** between words
- Remove any **leading or trailing spaces**

Example 1

```
Input:
"welcome to the jungle"

Output:
"jungle the to welcome"
```

Example 2

```
Input:
" amazing coding skills "

Output:
```

```
"skills coding amazing"
```

Brute Force Approach

Idea

Manually extract all words, store them, reverse the list of words, and then join them with a single space.

This approach handles:

- Multiple spaces
- Leading spaces
- Trailing spaces

Steps

1. Initialize an empty list `words`
2. Traverse the string character by character
3. Build words using consecutive non-space characters
4. Ignore extra spaces
5. Push each word into the list
6. Reverse the list of words
7. Join them using a single space

C++ Code (Brute Force)

```
#include <bits/stdc++.h>
```

```cpp
using namespace std;

class Solution {
public:
    string reverseWords(string s) {
        vector<string> words;
        string word = "";

        for (char ch : s) {
            if (ch != ' ') {
                word += ch;
            } else if (!word.empty()) {
                words.push_back(word);
                word = "";
            }
        }

        if (!word.empty()) {
            words.push_back(word);
        }

        reverse(words.begin(), words.end());

        string result = "";
        for (int i = 0; i < words.size(); i++) {
            result += words[i];
            if (i < words.size() - 1)
                result += " ";
        }

        return result;
    }
};

int main() {
    Solution obj;
    string s = " amazing coding skills ";
    cout << obj.reverseWords(s) << endl;
```

```
    return 0;
}
```

Complexity (Brute Force)

- **Time Complexity:** `O(N)`
- **Space Complexity:** `O(N)` (extra list to store words)

Optimal Approach (Right-to-Left Scan)

Idea

Instead of storing words and reversing later:

- Scan the string **from right to left**
- Extract each word and append it directly to the result

This avoids:

- Extra list
- Explicit reverse step

Steps

1. Start from the end of the string
2. Skip spaces
3. Identify a word by moving left until a space
4. Append the word to result

5. Add a space only when needed
6. Repeat until the beginning

C++ Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    string reverseWords(string s) {
        string result = "";
        int i = s.size() - 1;

        while (i >= 0) {
            while (i >= 0 && s[i] == ' ') //space skip
                i--;

            if (i < 0) break;

            int end = i;

            while (i >= 0 && s[i] != ' ')
                i--;

            string word = s.substr(i + 1, end - i);

            if (!result.empty())
                result += " ";

            result += word;
        }

        return result;
    }
};
```

```
int main() {
    Solution obj;
    string s = " amazing coding skills ";
    cout << obj.reverseWords(s) << endl;
    return 0;
}
```

Complexity (Optimal)

- **Time Complexity:** `O(N)`
- **Space Complexity:** `O(1)` (excluding output string)

Comparison Summary

Approach	Time	Space
Brute Force	`O(N)`	`O(N)`
Optimal	`O(N)`	`O(1)`

3. Largest Odd Number in a String

Problem Statement

You are given a string s representing a **large integer**.
Your task is to return the **largest-valued odd integer** (as a string) that can be formed as a **substring** of s.

Rules:

- The returned number **must be odd**
- It **must not have leading zeroes**
- The input string **may have leading zeroes**
- If no odd number exists, return an **empty string**

Example 1

```
Input:
s = "5347"

Output:
"5347"

Explanation:
Odd substrings → 5, 3, 53, 347, 5347
Largest odd number = 5347
```

Example 2

```
Input:
s = "0214638"

Output:
"21463"

Explanation:
Odd substrings → 1, 3, 21, 63, 463, 1463, 21463
Substrings starting with 0 are invalid
Largest valid odd number = 21463
```

Approach

Key Observations

1. **Odd number rule**
   → An odd number **must end with an odd digit** (`1, 3, 5, 7, 9`)

2. **Largest value rule**
   → Larger length ⇒ larger number
   → So we want the **longest possible substring** ending at an odd digit

3. **Leading zero rule**
   → We must **skip leading zeroes** in the final answer

Strategy

1. Scan the string from **right to left**
    - Find the **last odd digit**
    - This will be the **end** of the largest odd number
2. From the **start of the string**, skip all leading zeroes **up to that index**
3. Extract the substring between these two positions

Step-by-Step

For `s = "0214638"`

- Scan from right → last odd digit is `'3'` at index 5
- Skip leading zeroes → start from `'2'`
- Substring → `"21463"`

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    string largeOddNum(string& s) {
        int ind = -1;

        // Step 1: find last odd digit
        for (int i = s.length() - 1; i >= 0; i--) {
            if ((s[i] - '0') % 2 == 1) {
                ind = i;
                break;
            }
        }

        // If no odd digit found
        if (ind == -1) return "";

        // Step 2: skip leading zeroes
        int start = 0;
        while (start <= ind && s[start] == '0') {
            start++;
        }

        // Step 3: extract result
        return s.substr(start, ind - start + 1);
    }
};

int main() {
    Solution solution;
    string s = "0214638";
    cout << solution.largeOddNum(s) << endl;
    return 0;
```

```
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  (single pass from right + single pass from left)

- **Space Complexity:** `O(1)`
  (only indices used, output excluded)

4. Longest Common Prefix

Problem Statement

Given an array of strings, find the **longest common prefix** shared by **all** strings.
If no common prefix exists, return an empty string `""`.

Example 1

```
Input:

["flower", "flow", "flight"]


Output:

"fl"
```

All strings start with `"fl"`.

Example 2

```
Input:

["apple", "banana", "grape", "mango"]


Output:

""
```

No common starting sequence exists.

Approach (Sorting + Comparison)

Key Insight

After sorting strings **lexicographically**:

- The **first** and **last** strings will be the **most different**
- Any common prefix shared by **all** strings must also be shared by these two

So instead of comparing all strings:
👉 **Only compare the first and last string after sorting**

Step-by-Step Algorithm

1. If the array is empty → return `""`
2. Sort the array of strings
3. Take:

- `first = str[0]`
- `last = str[n-1]`

4. Compare characters from index `0` onwards
5. Stop when characters differ
6. Return the matched prefix

Why This Works

Sorting ensures:

- Strings with **smallest** and **largest** lexicographical order are farthest apart
- If they match up to some point, **all middle strings must also match**

C++ Implementation

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    string longestCommonPrefix(vector<string>& str) {

        if (str.empty()) return "";


        // Sort strings lexicographically
```

```
        sort(str.begin(), str.end());

        string first = str[0];
        string last = str[str.size() - 1];

        string ans = "";
        int minLen = min(first.size(), last.size());

        // Compare characters of first and last string
        for (int i = 0; i < minLen; i++) {
            if (first[i] != last[i])
                break;
            ans += first[i];
        }

        return ans;
    }
};

int main() {
    vector<string> input = {"interview", "internet", "internal",
"interval"};
    Solution sol;
```

```
    cout << sol.longestCommonPrefix(input) << endl;

    return 0;

}
```

Complexity Analysis

- **Time Complexity:**
  `O(N log N + M)`
    - Sorting → `O(N log N)`
    - Prefix comparison → `O(M)`
      (`N` = number of strings, `M` = min string length)
- **Space Complexity:**
  `O(M)` for storing the prefix

5. Isomorphic String

Problem Statement

Given two strings `s` and `t`, check whether they are **isomorphic**.

Two strings are isomorphic if:

- Each character in `s` can be replaced to get `t`
- **Same character must always map to the same character**

- **No two different characters can map to the same character**
- Order of characters must remain the same

Example 1

```
Input:
s = "paper"
t = "title"

Output:
true
```

Mapping:

```
p → t
a → i
p → t   (same as before ✔)
e → l
r → e
```

Mapping is consistent → **isomorphic**

Example 2

```
Input:
s = "foo"
t = "bar"

Output:
false
```

Mapping:

```
f → b
o → a
```

`o → r` ❌ `(conflict)`

Same character o maps to two different characters → **not isomorphic**

Key Idea (Very Important)

Instead of directly storing character-to-character mapping, we compare **patterns of occurrence**.

If two characters:

- First appeared at the same index
- Last appeared at the same index

Then they follow the **same structure**.

Optimal Approach (Pattern Matching using Last Seen Index)

Core Logic

- Use two arrays of size 256 (for all ASCII characters)
- Store **last seen position** of each character
- Traverse both strings together
- At every index `i`:
    - If last seen position of `s[i]` ≠ last seen position of `t[i]` → mapping conflict → return false
    - Otherwise, update both positions

Why `i + 1`?

- Default value is `0`
- Using `i + 1` avoids confusion with unseen characters

Step-by-Step Algorithm

1. If lengths of `s` and `t` differ → return false
2. Create two arrays `m1` and `m2` of size `256`, initialized to `0`
3. Traverse both strings from left to right
4. At index `i`:
    - If `m1[s[i]] != m2[t[i]]` → return false

Update:

```
 m1[s[i]] = i + 1
m2[t[i]] = i + 1
```

    -
5. If loop completes → return true

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isomorphicString(string s, string t) {
        // If lengths differ, they cannot be isomorphic
        if (s.size() != t.size()) return false;
```

```
        int m1[256] = {0};
        int m2[256] = {0};

        for (int i = 0; i < s.size(); i++) {
            // If last seen positions do not match → conflict
            if (m1[s[i]] != m2[t[i]])
                return false;

            // Update last seen position
            m1[s[i]] = i + 1;
            m2[t[i]] = i + 1;
        }
        return true;
    }
};

int main() {
    Solution solution;
    string s = "paper";
    string t = "title";

    cout << (solution.isomorphicString(s, t) ? "true" : "false") <<
endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N)$
  Single pass through both strings

- **Space Complexity:** $O(1)$
  Fixed-size arrays (256), constant space

- No hashmap overhead

- Very fast
- Directly checks **structure**, not characters
- Works for all ASCII characters

One-Line Intuition

> If two strings follow the **same pattern of repetition**, they are isomorphic.

6. Check if One String is Rotation of Another

Problem Statement

Given two strings `s` and `goal`, return **true** if `s` can be converted into `goal` using **left rotations**. A left rotation means moving the **first character to the end**.

Example 1

```
Input:
s = "rotation"
goal = "tionrota"

Output:
true
```

After rotations:

`rotation → otationr → tationro → ationrot → tionrota`

Example 2

```
Input:
s = "hello"
goal = "lohelx"

Output:
false
```

Extra character x makes rotation impossible.

Brute Force Approach

Idea

- Generate **all rotations** of s
- Compare each rotation with `goal`
- If any match → return true

Algorithm

1. If lengths are different → return false
2. For `i = 0 to n-1`:
    - Create rotation: `s.substr(i) + s.substr(0, i)`
    - If rotation == goal → return true

3. Return false

C++ Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool rotateString(string& s, string& goal) {
        if (s.length() != goal.length()) return false;

        for (int i = 0; i < s.length(); i++) {
            string rotated = s.substr(i) + s.substr(0, i);
            if (rotated == goal)
                return true;
        }
        return false;
    }
};

int main() {
    Solution sol;
    string s = "rotation";
    string goal = "tionrota";
    cout << (sol.rotateString(s, goal) ? "true" : "false") << endl;
    return 0;
}
```

Complexity

- **Time:** $O(N^2)$
- **Space:** $O(N)$

Optimal Approach (Key Trick)

Core Observation

If `goal` is a rotation of `s`, then:

> `goal` **must be a substring of** `s + s`

Example:

```
s = "abcde"
s+s = "abcdeabcde"

All rotations exist inside this string
```

Why This Works

- Rotation does not change order, only shifts starting point
- Doubling `s` contains **all possible rotations**
- Substring check confirms existence

Algorithm

1. If lengths differ → return false
2. Create `doubled = s + s`
3. If `goal` is a substring of `doubled` → return true
4. Else → return false

C++ Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool rotateString(string& s, string& goal) {
        if (s.length() != goal.length()) return false;
        string doubled = s + s;
        return doubled.find(goal) != string::npos;
    }
};

int main() {
    Solution sol;
    string s = "rotation";
    string goal = "tionrota";
    cout << (sol.rotateString(s, goal) ? "true" : "false") << endl;
    return 0;
}
```

Complexity

- **Time:** `O(N)`
- **Space:** `O(N)`

One-Line Intuition

If `goal` is a rotation of `s`, it must appear somewhere inside `s + s`.

7. Check if Two Strings Are Anagrams

Problem Statement

Given two strings, check whether they are **anagrams** of each other.

Two strings are anagrams if:

- They contain **exactly the same characters**
- Each character appears the **same number of times**
- Order does **not** matter

Example 1

```
Input:  "CAT", "ACT"
Output: true
```

Both strings contain: C, A, T (same count)

Example 2

```
Input:  "RULES", "LESRT"
Output: false
```

Character counts differ (U vs T)

Brute Force Approach (Sorting)

Idea

If two strings are anagrams, then after sorting:

- Both strings will look **exactly the same**

Algorithm

1. If lengths differ → return false
2. Sort both strings
3. Compare characters one by one
4. If all match → return true, else false

C++ Code (Sorting)

```
#include <bits/stdc++.h>
using namespace std;

bool CheckAnagrams(string str1, string str2) {
    if (str1.length() != str2.length())
        return false;

    sort(str1.begin(), str1.end());
    sort(str2.begin(), str2.end());

    for (int i = 0; i < str1.length(); i++) {
        if (str1[i] != str2[i])
            return false;
    }
    return true;
}

int main() {
    string a = "INTEGER";
    string b = "TEGERNI";
```

```
    cout << (CheckAnagrams(a, b) ? "True" : "False") << endl;
    return 0;
}
```

Complexity

- **Time:** `O(N log N)` (due to sorting)
- **Space:** `O(1)` (in-place sort)

Optimal Approach (Frequency Counting)

Key Observation

Anagrams depend on **character frequency**, not order.

So:

- Count characters in first string
- Subtract counts using second string
- If all counts become zero → anagram

Algorithm

1. If lengths differ → return false
2. Create frequency array of size 26
3. Traverse first string → increment count
4. Traverse second string → decrement count

5. If any count ≠ 0 → return false
6. Else → return true

C++ Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

bool CheckAnagrams(string str1, string str2) {
    if (str1.length() != str2.length())
        return false;

    int freq[26] = {0};

    for (char c : str1)
        freq[c - 'A']++;

    for (char c : str2)
        freq[c - 'A']--;

    for (int i = 0; i < 26; i++) {
        if (freq[i] != 0)
            return false;
    }
    return true;
}

int main() {
    string a = "INTEGER";
    string b = "TEGERNI";
    cout << (CheckAnagrams(a, b) ? "True" : "False") << endl;
    return 0;
}
```

- **Time:** $O(N)$
- **Space:** $O(1)$ (fixed-size array)

Why Optimal Approach Is Better

- No sorting overhead
- Linear time
- Directly checks the core requirement (frequency)

One-Line Intuition

> Two strings are anagrams if **every character appears the same number of times** in both.

8. Sort Characters by Frequency

Problem Statement

Given a string `s`, return the **unique characters** sorted by:

1. **Higher frequency first**
2. If frequencies are equal → **alphabetical order**

Example 1

```
Input:  "tree"
Output: ['e', 'r', 't']
```

Frequency:

```
e → 2
r → 1
t → 1
```

Same frequency → alphabetical (`r` before `t`)

Example 2

```
Input:  "raaaajj"
Output: ['a', 'j', 'r']
```

Frequency:

```
a → 4
j → 2
r → 1
```

Approach (Frequency + Sorting)

Core Idea

- Count how many times each character appears
- Sort characters based on:
    - **frequency (descending)**
    - **character (ascending)** if frequency ties

Step-by-Step Algorithm

1. Create a frequency array of size 26
2. Count frequency of each character
3. Store `(frequency, character)` pairs
4. Sort using a custom comparator:
    - Higher frequency first
    - Alphabetical order if equal
5. Collect characters with frequency > 0

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    // Comparator: frequency desc, character asc
    static bool cmp(pair<int, char> a, pair<int, char> b) {
        if (a.first != b.first)
            return a.first > b.first;
        return a.second < b.second;
    }

public:
    vector<char> frequencySort(string& s) {
        pair<int, char> freq[26];

        // Initialize
        for (int i = 0; i < 26; i++)
```

```
            freq[i] = {0, char('a' + i)};

        // Count frequency
        for (char c : s)
            freq[c - 'a'].first++;

        // Sort by frequency and alphabet
        sort(freq, freq + 26, cmp);

        // Collect result
        vector<char> ans;
        for (int i = 0; i < 26; i++) {
            if (freq[i].first > 0)
                ans.push_back(freq[i].second);
        }

        return ans;
    }
};

int main() {
    Solution sol;
    string s = "tree";
    vector<char> res = sol.frequencySort(s);

    for (char c : res)
        cout << c << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N + 26 \log 26)$ → effectively $O(N)$
- **Space Complexity:** $O(26)$ → constant space

9. Maximum Nesting Depth of Parenthesis

Problem Statement

Given a **valid parentheses string** s, return its **maximum nesting depth**.

Nesting depth = maximum number of parentheses that are **open at the same time**.

Example 1

```
Input:
"(1+(2*3)+((8)/4))+1"

Output:
3
```

At digit 8:

```
((8))
 ↑ ↑ ↑   → depth = 3
```

Example 2

```
Input:
"(1)+((2))+(((3)))"

Output:
3
```

Deepest nesting happens at 3.

Key Intuition

Think of parentheses like **going up and down floors**:

- `'('` → go **one level deeper**
- `')'` → come **one level back**
- Track the **maximum level reached**

Step-by-Step Algorithm

1. Initialize:
    - `depth = 0` → current nesting level
    - `maxDepth = 0` → answer
2. Traverse the string character by character
3. If character is `'('`:
    - Increase `depth`
    - Update `maxDepth`
4. If character is `')'`:
    - Decrease `depth`
5. Ignore all other characters
6. Return `maxDepth`

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maxDepth(string s) {
        int depth = 0;
        int maxDepth = 0;

        for (char c : s) {
            if (c == '(') {
                depth++;
                maxDepth = max(maxDepth, depth);
            }
            else if (c == ')') {
                depth--;
            }
        }
        return maxDepth;
    }
};

int main() {
    Solution sol;
    string s = "(1+(2*3)+((8)/4))+1";
    cout << sol.maxDepth(s) << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Single traversal of the string

- **Space Complexity:** O(1)
  Only two integer variables used

Dry Run (Quick)

String: "((a)+(b))"

Char	depth	maxDepth
(	1	1
(	2	2
a	2	2
)	1	2
+	1	2
(	2	2
b	2	2
)	1	2
)	0	2

Answer = **2**

One-Line Intuition

> Maximum nesting depth is the highest number of '(' opened at any moment.

10. Roman Numerals to Integer

Problem Statement

Roman numerals use the following symbols:

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Normally, values are **added from left to right**.
But in some cases, **subtraction** is used:

- `I` before `V` or `X` → 4, 9
- `X` before `L` or `C` → 40, 90
- `C` before `D` or `M` → 400, 900

Your task: **convert a Roman numeral string into an integer**.

Example 1

```
Input:  "LVIII"
Output: 58
```

Explanation:

```
L = 50
V = 5
III = 3
Total = 58
```

Example 2

```
Input:  "MCMXCIV"
Output: 1994
```

Explanation:

```
M  = 1000
CM = 900
XC = 90
IV = 4
Total = 1994
```

Key Idea (Most Important)

While scanning the string:

- If **current value < next value** → subtract it
- Else → add it

Why?

- Smaller numeral before a larger one means subtraction (`IV`, `IX`, etc.)

Step-by-Step Algorithm

1. Create a map of Roman symbols to values
2. Initialize `result = 0`
3. Traverse the string from **left to right**
4. At index `i`:
    - If `i + 1 < n` and `value(s[i]) < value(s[i+1])`
        - Subtract `value(s[i])`
    - Else
        - Add `value(s[i])`
5. Return `result`

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int romanToInt(string s) {
        unordered_map<char, int> roman = {
            {'I', 1}, {'V', 5}, {'X', 10},
            {'L', 50}, {'C', 100}, {'D', 500}, {'M', 1000}
        };

        int result = 0;
        int n = s.size();

        for (int i = 0; i < n; i++) {
```

```
            // If next character exists and current value is smaller
            if (i + 1 < n && roman[s[i]] < roman[s[i + 1]]) {
                result -= roman[s[i]];
            } else {
                result += roman[s[i]];
            }
        }
        return result;
    }
};

int main() {
    Solution sol;
    cout << sol.romanToInt("LVIII") << endl;    // 58
    cout << sol.romanToInt("MCMXCIV") << endl;  // 1994
    return 0;
}
```

Dry Run (Very Important)

Input: "MCMXCIV"

Index	Char	Value	Next	Action	Result
0	M	1000	C	add	1000
1	C	100	M	subtract	900
2	M	1000	X	add	1900
3	X	10	C	subtract	1890
4	C	100	I	add	1990
5	I	1	V	subtract	1989
6	V	5	—	add	1994

Complexity Analysis

- **Time Complexity:** `O(N)`
  Single pass through the string

- **Space Complexity:** `O(1)`
  Fixed-size map (7 symbols)

11. Implement `atoi` (String to Integer)

Problem Statement

Implement `myAtoi(string s)` that converts a string into a **32-bit signed integer**, following these rules **in order**:

1. **Ignore leading spaces**
2. **Check sign** (`-` or `+`, default is positive)
3. **Read digits** until a non-digit appears
4. **If no digits are read → return 0**
5. **Clamp result** to range
   [−231, 231−1][-2^{31},\; 2^{31}-1][−231,231−1]

Examples

Example 1

```
Input:  "1337c0d3"
Output: 1337
```

Stops reading at `'c'`.

Example 2

```
Input:  "words and 987"
Output: 0
```

First character is non-digit → no conversion.

Core Idea (Very Important)

We **scan the string once**, building the number digit by digit, while carefully:

- skipping spaces
- handling sign
- stopping at non-digits
- checking overflow **before it happens**

Step-by-Step Algorithm

1. Initialize:
    - `i = 0` (index)
    - `sign = 1`
    - `result = 0` (use `long` to detect overflow)
2. **Skip leading spaces**

3. **Check sign**
    - `'-'` → `sign = -1`
    - `'+'` → `sign = 1`
4. **Read digits**
    - `result = result * 10 + digit`
    - After each step, check:
        - if `sign * result > INT_MAX` → return `INT_MAX`
        - if `sign * result < INT_MIN` → return `INT_MIN`
5. Return `sign * result`

Why Overflow Check Is Needed Early

If we wait until the end, the number may already overflow.

So we **check during construction**, not after.

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int myAtoi(string s) {
        int i = 0, sign = 1;
        long res = 0;

        // 1. Skip leading whitespaces
```

```
        while (i < s.size() && s[i] == ' ')
            i++;

        // 2. Check sign
        if (i < s.size() && (s[i] == '+' || s[i] == '-')) {
            if (s[i] == '-') sign = -1;
            i++;
        }

        // 3. Convert digits
        while (i < s.size() && isdigit(s[i])) {
            res = res * 10 + (s[i] - '0');

            // 4. Clamp if overflow
            if (sign * res > INT_MAX)
                return INT_MAX;
            if (sign * res < INT_MIN)
                return INT_MIN;

            i++;
        }

        return sign * res;
    }
};

int main() {
    Solution sol;
    cout << sol.myAtoi("   -42") << endl;      // -42
    cout << sol.myAtoi("4193 with words") << endl; // 4193
    cout << sol.myAtoi("words and 987") << endl;   // 0
    return 0;
}
```

Dry Run (Important)

Input: " -42"

```
Skip spaces → index at '-'
sign = -1
Read digits: 4 → 42
Result = -42
```

Input: "91283472332"

```
Number exceeds INT_MAX
Return 2147483647
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Single pass through the string

- **Space Complexity:** `O(1)`
  Only a few variables

Common Mistakes to Avoid

❌ Not skipping leading spaces
❌ Not handling sign
❌ Parsing digits after non-digit
❌ Checking overflow only at the end

One-Line Intuition

> Skip spaces → read sign → build number digit by digit → stop early if overflow.

12. Count Number of Substrings with Exactly K Distinct Characters

---

Problem Statement

You are given:

- a string s
- an integer k

Return the **number of substrings** of s that contain **exactly k distinct characters**.

Example 1

```
Input:
s = "pqpqs", k = 2

Output:
7
```

Valid substrings:

```
"pq", "pqp", "pqpq", "qp", "qpq", "pqs", "qs"
```

Example 2

```
Input:
s = "abcbaa", k = 3

Output:
```

5

Valid substrings:

`"abc", "abcb", "abcba", "bcba", "cbaa"`

Important Insight (Very Important)

Directly counting substrings with **exactly K** distinct characters is hard.

So we use this trick:

Exactly K = At Most K − At Most (K − 1)

Why?

- Substrings with **at most K** distinct characters include:
    - substrings with 1, 2, …, K distinct characters
- Subtracting **at most (K−1)** removes all substrings with fewer than K distinct characters
- What remains → **exactly K**

Helper Function: `atMostKDistinct(s, k)`

This function returns the number of substrings with **at most k distinct characters**.

Sliding Window Logic (Core Concept)

We use:

- `left` pointer → start of window

- `right` pointer → end of window
- a frequency map to count characters in the window

Window Rules

- Expand window by moving `right`
- If distinct characters exceed k, shrink window from `left`

For every valid window, add

`(right - left + 1)`

- because all substrings ending at `right` and starting from `left…right` are valid

---

Step-by-Step Algorithm

atMostKDistinct(s, k)

1. Initialize `left = 0`, `result = 0`
2. Traverse string with `right`
3. Add `s[right]` to frequency map
4. If distinct characters > k:
    - shrink window from `left`
5. Add `(right - left + 1)` to result
6. Return result

Exactly K Distinct

```
answer = atMostKDistinct(s, k)
        - atMostKDistinct(s, k - 1)
```

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Count substrings with at most k distinct characters
int atMostKDistinct(string s, int k) {
    int left = 0, res = 0;
    unordered_map<char, int> freq;

    for (int right = 0; right < s.size(); right++) {
        freq[s[right]]++;

        // Shrink window if distinct characters exceed k
        while (freq.size() > k) {
            freq[s[left]]--;
            if (freq[s[left]] == 0)
                freq.erase(s[left]);
            left++;
        }

        // Count valid substrings ending at 'right'
        res += (right - left + 1);
    }
    return res;
}

// Count substrings with exactly k distinct characters
int countSubstrings(string s, int k) {
    return atMostKDistinct(s, k) - atMostKDistinct(s, k - 1);
}

int main() {
```

```
    string s = "pqpqs";
    int k = 2;
    cout << countSubstrings(s, k) << endl; // Output: 7
    return 0;
}
```

Dry Run (Small Insight)

For `s = "pqpqs", k = 2`:

- `atMostKDistinct(2)` counts substrings with 1 or 2 distinct chars
- `atMostKDistinct(1)` counts substrings with only 1 distinct char
- Difference → substrings with **exactly 2**

Complexity Analysis

- **Time Complexity:** `O(N)`
    - Each character enters and leaves the window once
- **Space Complexity:** `O(1)`
    - Map size bounded by alphabet size (≤ 26 for lowercase)

One-Line Intuition

> Count substrings with at most K distinct characters, subtract those with at most K−1.

13. Longest Palindromic Substring

Problem Statement

Given a string s, return the **longest substring** of s that is a **palindrome**.

A palindrome reads the same **forward and backward**.

Examples

Example 1

```
Input:  "babad"
Output: "bab"
```

("aba" is also valid)

Example 2

```
Input:  "cbbd"
Output: "bb"
```

Core Idea (Very Important)

Every palindrome has a **center**.

There are **two types** of palindromes:

1. **Odd length** → center at a character
   Example: "bab" (center = a)

2. **Even length** → center between two characters
   Example: "bb" (center between b and b)

👉 So for every index, we try **both cases** and expand outward.

How Expansion Works

From a center:

- Move `left` to the left
- Move `right` to the right
- Keep expanding **while characters match**

When they stop matching → palindrome ends.

Step-by-Step Algorithm

1. Initialize `start = 0`, `end = 0`
2. For each index `i` in the string:
    - Check **odd-length palindrome** (center = `i`)
    - Check **even-length palindrome** (center = `i, i+1`)
3. Take the **maximum length** from both
4. Update `start` and end if a longer palindrome is found
5. Return `s.substr(start, end - start + 1)`

Why `start` and end Formula Works

For a palindrome of length `len` centered at `i`:

```
start = i - (len - 1) / 2
```

```
end   = i + len / 2
```

This formula works for **both odd and even lengths**.

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    string longestPalindrome(string s) {
        int start = 0, end = 0;

        for (int i = 0; i < s.length(); i++) {
            int len1 = expandFromCenter(s, i, i);     // odd
            int len2 = expandFromCenter(s, i, i + 1); // even
            int len = max(len1, len2);

            if (len > end - start) {
                start = i - (len - 1) / 2;
                end   = i + len / 2;
            }
        }
        return s.substr(start, end - start + 1);
    }

private:
    int expandFromCenter(const string& s, int left, int right) {
        while (left >= 0 && right < s.length() && s[left] == s[right])
{
            left--;
            right++;
        }
        return right - left - 1;
    }
};
```

```
int main() {
    Solution sol;
    cout << sol.longestPalindrome("babad") << endl;
    return 0;
}
```

Dry Run (Quick)

String: `"cbbd"`

- `i = 1`
- Even center `(1,2)` → `"bb"`
- Length = 2 → longest so far

Answer = `"bb"`

Complexity Analysis

- **Time Complexity:** `O(N²)`
    - Each index expands up to N
- **Space Complexity:** `O(1)`
    - No extra data structures

One-Line Intuition

Try every index as a center and expand outward to find the longest palindrome.

14. Sum of Beauty of All Substrings

Problem Recap

The **beauty** of a string =

`(max frequency of any character) - (min frequency of any character)`

👉 **Ignore characters with zero frequency**

You need to compute the **sum of beauty values of all substrings** of a given string `s`.

Key Observations

1. Substrings of length **1** always have beauty **0**
   (only one character → max = min)

2. Beauty depends only on **character frequencies**, not order.

3. Since the string contains lowercase letters, frequency size is at most **26**.

Approach (Brute Force but Optimal Enough)

We fix a **starting index `i`**
Then extend the substring to the right using **ending index `j`**

For each fixed `i`:

- Maintain a frequency array/map
- As `j` increases, update frequency of `s[j]`
- Compute:

    - `maxi` = maximum frequency
    - `mini` = minimum **non-zero** frequency
- Add (`maxi - mini`) to the answer

This avoids recomputing frequencies from scratch.

Why This Works

- Each substring is considered exactly once
- Frequency update is incremental
- Checking max/min over at most 26 characters → constant time

---

C++ Implementation (Clean & Clear)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int beautySum(string s) {
        int n = s.size();
        int ans = 0;

        // Fix starting point
        for (int i = 0; i < n; i++) {
            vector<int> freq(26, 0);

            // Extend substring
            for (int j = i; j < n; j++) {
                freq[s[j] - 'a']++;
```

```
                int maxi = 0;
                int mini = INT_MAX;

                // Calculate beauty
                for (int k = 0; k < 26; k++) {
                    if (freq[k] > 0) {
                        maxi = max(maxi, freq[k]);
                        mini = min(mini, freq[k]);
                    }
                }

                ans += (maxi - mini);
            }
        }
        return ans;
    }
};

int main() {
    Solution sol;
    cout << sol.beautySum("xyx") << endl;      // Output: 1
    cout << sol.beautySum("aabcbaa") << endl; // Output: 17
    return 0;
}
```

Dry Run (Short Example)

Input: "xyx"

Substrings and beauty:

- "x" → 0
- "y" → 0
- "x" → 0

- "xy" → x:1, y:1 → 0
- "yx" → y:1, x:1 → 0
- "xyx" → x:2, y:1 → **1**

✅ Total = **1**

Complexity Analysis

- **Time Complexity:**
  - Outer loop: $O(n)$
  - Inner loop: $O(n)$
  - Frequency scan: $O(26)$
    👉 Overall: **$O(n^2)$**
- **Space Complexity:**
  - Frequency array of size 26 → **O(1)**

One-Line Intuition

Fix a start, grow the substring, track frequencies, and keep adding (max − min).

15. Reverse Every Word in a String

Problem Statement

Given an input string containing upper-case and lower-case letters, digits, and spaces (' '), reverse the **order of words** in the string.

- A word is defined as a sequence of non-space characters.
- Words are separated by at least one space.
- The output should:
    - have **no leading or trailing spaces**
    - have **exactly one space** between words

Examples

Example 1

Input

```
s = "welcome to the jungle"
```

Output

```
"jungle the to welcome"
```

Example 2

Input

```
s = " amazing coding skills "
```

Output

```
"skills coding amazing"
```

Brute Force Approach

Algorithm

1. Traverse the string and extract all words while ignoring extra spaces.
2. Store each word in a list/vector.
3. Reverse the list of words.
4. Join the words using a single space.

Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    string reverseWords(string s) {
        vector<string> words;
        int n = s.size();
        int i = 0;

        while (i < n) {
            // Skip spaces
            while (i < n && s[i] == ' ') i++;
            if (i >= n) break;

            int start = i;
            while (i < n && s[i] != ' ') i++;
```

```
            words.push_back(s.substr(start, i - start));
        }

        // Reverse words
        reverse(words.begin(), words.end());

        // Join with single space
        string ans = "";
        for (int i = 0; i < words.size(); i++) {
            ans += words[i];
            if (i != words.size() - 1) ans += " ";
        }

        return ans;
    }
};

int main() {
    Solution sol;
    string s = " amazing coding skills ";
    cout << sol.reverseWords(s);
    return 0;
}
```

Complexity Analysis (Brute Force)

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(n)$ (stores words separately)

Optimal Approach

Algorithm

1. Reverse the entire string.
2. Traverse the reversed string word by word.
3. Reverse each word individually to restore correct character order.
4. While processing, skip extra spaces and ensure only one space between words.
5. Remove any trailing space.

Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    void reversePart(string &s, int l, int r) {
        while (l < r) {
            swap(s[l++], s[r--]);
        }
    }

public:
    string reverseWords(string s) {
        int n = s.size();

        // Reverse entire string
        reversePart(s, 0, n - 1);

        int i = 0, j = 0;

        while (j < n) {
            // Skip spaces
            while (j < n && s[j] == ' ') j++;
            if (j >= n) break;

            int start = i;
```

```
            // Copy characters of word
            while (j < n && s[j] != ' ') {
                s[i++] = s[j++];
            }
//j hamesha i ke barabar ya aage hota hai
// → overwrite ka risk nahi hota.


            // Reverse current word
            reversePart(s, start, i - 1);

            // Add single space
            s[i++] = ' ';
        }

        // Remove trailing space
        if (i > 0) i--;

        return s.substr(0, i);
    }
};

int main() {
    Solution sol;
    string s = " amazing coding skills ";
    cout << sol.reverseWords(s);
    return 0;
}
```

Complexity Analysis (Optimal)

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$ (in-place)

HARD PROBLEMS

16. Minimum Number of Bracket Reversals Needed to Make an Expression Balanced

Given a string s consisting only of opening ' (' and closing ') ' brackets, you have to find the **minimum number of reversals** required to make the expression **balanced**.

A reversal means:

- ' (' can be changed to ') '
- ') ' can be changed to ' ('

If it is **not possible** to balance the string, return **-1**.

A balanced expression means every opening bracket has a matching closing bracket in the correct order.

Example
s = ")(())((("
One possible balanced form is " ((( )) ( )) "
Minimum reversals required = 3

s = "((()(((()(()((("
Total length is odd, so it is impossible to balance
Output = -1

Approach

Using Counting of Unmatched Brackets

Algorithm

1. If the length of the string is **odd**, it is impossible to balance because brackets must form pairs. Return `-1`.
2. Initialize two counters:
    - `openBrackets` → count of unmatched `'('`
    - `closeBrackets` → count of unmatched `')'`
3. Traverse the string character by character:
    - If the character is `'('`, increase `openBrackets`.
    - If the character is `')'`:
        - If there is an unmatched `'('`, match it by decreasing `openBrackets`.
        - Otherwise, this `')'` is unmatched, so increase `closeBrackets`.
4. After traversal, some brackets may remain unmatched.
5. Minimum reversals required:
    - `(openBrackets + 1) / 2` reversals to fix unmatched `'('`
    - `(closeBrackets + 1) / 2` reversals to fix unmatched `')'`
6. Return the sum of both.

Why this works

- Two unmatched `'('` can be fixed with one reversal.
- Same logic applies to unmatched `')'`.
- Integer division handles both even and odd counts correctly.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   int minReversalsToBalance(string expression) {
      int openBrackets = 0;
      int closeBrackets = 0;

      for (char ch : expression) {
         if (ch == '(') {
            openBrackets++;
         } else {
            if (openBrackets > 0) {
               openBrackets--;
            } else {
               closeBrackets++;
            }
         }
      }
```

```
        if ((openBrackets + closeBrackets) % 2 != 0)
            return -1;

        return (openBrackets + 1) / 2 + (closeBrackets + 1) / 2;
    }
};

int main() {
    string expression = ")(())(((";
    Solution solver;
    cout << solver.minReversalsToBalance(expression);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  The string is traversed once.
- **Space Complexity:** O(1)
  Only constant extra variables are used.

17. Count and Say

The **count-and-say sequence** is a sequence of strings where each term is generated by **describing the previous term**.

Rules:

- `countAndSay(1) = "1"`
- `countAndSay(n)` is formed by applying **run-length encoding** on `countAndSay(n-1)`

Run-length encoding means counting **consecutive identical digits** and writing:
`count + digit`

Example
Input: `n = 4`

Sequence:

- `countAndSay(1) = "1"`
- `countAndSay(2) = "11"` → one 1
- `countAndSay(3) = "21"` → two 1s
- `countAndSay(4) = "1211"` → one 2, one 1

Output: `"1211"`

Approach

Algorithm

1. Start with the base string `"1"`.
2. Repeat the following process `n-1` times:
    - Traverse the current string from left to right.
    - Count how many times the current character repeats consecutively.
    - Append `count` followed by the character to form the next string.
3. Update the current string with the newly formed string.
4. After completing all iterations, return the final string.

Example Walkthrough
Previous term = `"21"`

- One 2 → `"12"`
- One 1 → `"11"`
  Next term = `"1211"`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   string countAndSay(int n) {
      string result = "1";

      for (int i = 1; i < n; i++) {
         string current = "";
```

```
            int count = 1;

            for (int j = 1; j < result.size(); j++) {
                if (result[j] == result[j - 1]) {
                    count++;
                } else {
                    current += to_string(count) + result[j - 1];
                    count = 1;
                }
            }

            current += to_string(count) + result.back();
            result = current;
        }
        return result;
    }
};

int main() {
    Solution solver;
    int n = 4;
    cout << solver.countAndSay(n);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n \cdot 2^n)$
  There are n iterations and the length of the string can grow exponentially.
- **Space Complexity:** $O(2^n)$
  The final generated string can have exponential length.

18. Hashing in Strings

Introduction:

In this article, we will be discussing the ***Hashing in Strings.*** Hashing refers to the process of converting a string into a numeric value, called a hash value, using a mathematical formula. It helps in comparing strings, searching efficiently, and solving complex problems like pattern matching.

Need of Hashing:

Hashing is essential as it helps in following operations:

1. **Efficient Comparisons**: Comparing large strings is slow. Hash values allow comparison in constant time.
2. **Faster Lookups**: Hash tables like dictionaries or unordered maps use hash values to store and retrieve strings quickly.
3. **Detect Duplicates or Anagrams**: Hashing helps in checking duplicates or determining anagram groups efficiently.
4. **Pattern Matching**: Algorithms like Rabin-Karp use string hashing to find substrings in linear time.

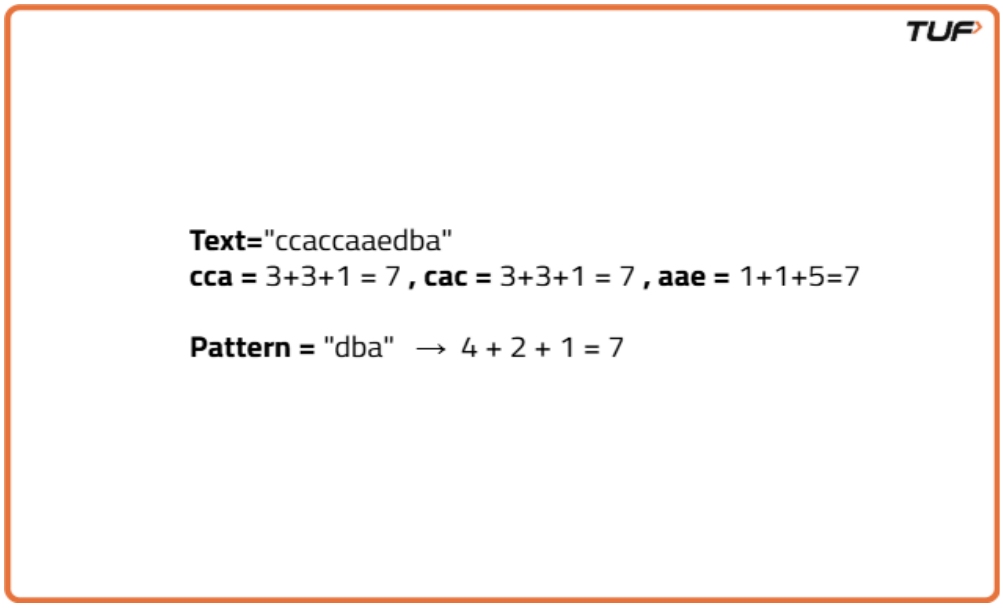
How Does String Hashing Work?:

String hashing treats each character as a number and combines them using a mathematical formula. A common formula is **Polynomial Rolling Hash**. Using this hash value is calculated as follows:
Hash(s) = (s[0] * p^0 + s[1] * p^1 + s[2] * p^2 + ... + s[n-1] * p^(n-1)) % m

where, s[i] is the character's ASCII or mapped value (like 'a' = 1),
p is a prime base (commonly 31 or 53) and
m is a large prime modulus to avoid overflow and reduce collisions.

Handling Collisions:

Two different strings can have the same hash, called a collision. To reduce this, we use a large m (like 1e9+7). We can also use double hashing (combine two different hash functions) or use a good base (p) that spreads values well.



Example:

Calculate Hash value of string s = "abc".

Solution:

Let us assume p to be 31 and m to be 1e9+9.

Now, we can map characters as following: 'a' = 1, 'b' = 2 and 'c' = 3.

Thus, Hash(s) = (1 * 31^0 + 2 * 31^1 + 3 * 31^2) % 1e9+9

Hash(s) = 2946.

Therefore, "abc" gets hashed to value 2946.

19. Rabin Karp Algorithm – Pattern Searching

Given a string `text` and a string `pattern`, you need to find **all starting indices** where `pattern` occurs in `text`.
If the pattern does not occur, return an empty list.

The Rabin-Karp algorithm uses **hashing with a sliding window** to efficiently search for patterns in a string.

Example
text = `"ababcabcababc"`
pattern = `"abc"`

The pattern `"abc"` appears starting at indices `2`, `5`, and `10`.
Output = `[2, 5, 10]`

Approach

Rabin-Karp Algorithm (Rolling Hash)

Algorithm

1. If the pattern length is greater than the text length, return an empty list.
2. Choose:
    - A base value (number of possible characters, usually 256).
    - A prime modulus to reduce collisions.
3. Compute the hash of the `pattern`.
4. Compute the hash of the first window of `text` having the same length as the pattern.
5. Slide the window over the text one character at a time:
    - If the current window hash matches the pattern hash:
        - Compare characters one by one to confirm the match.
    - Update the window hash using the rolling hash formula:
        - Remove the leftmost character.
        - Add the next character to the window.

6. Store all indices where a confirmed match is found.
7. Return the list of indices.

Why character comparison is needed after hash match
Different strings can sometimes have the same hash value (hash collision).
So, whenever hashes match, we verify by direct comparison.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   vector<int> rabinKarp(string text, string pattern) {
      vector<int> result;

      int n = text.length();
      int m = pattern.length();
      if (m > n) return result;

      int base = 256;
      int mod = 101;

      int patternHash = 0;
      int windowHash = 0;
      int h = 1;

      for (int i = 0; i < m - 1; i++) {
         h = (h * base) % mod;
      }

      for (int i = 0; i < m; i++) {
         patternHash = (base * patternHash + pattern[i]) % mod;
         windowHash = (base * windowHash + text[i]) % mod;
      }

      for (int i = 0; i <= n - m; i++) {
         if (patternHash == windowHash) {
            bool match = true;
            for (int j = 0; j < m; j++) {
               if (text[i + j] != pattern[j]) {
                  match = false;
                  break;
               }
```

```
            }
            if (match) result.push_back(i);
        }

        if (i < n - m) {
            windowHash = (base * (windowHash - text[i] * h)
                    + text[i + m]) % mod;
            if (windowHash < 0) windowHash += mod;
        }
    }

    return result;
  }
};

int main() {
  string text = "ababcabcababc";
  string pattern = "abc";

  Solution sol;
  vector<int> ans = sol.rabinKarp(text, pattern);

  for (int idx : ans) {
    cout << idx << " ";
  }
  return 0;
}
```

Complexity Analysis

- **Time Complexity:**
  Average case: $O(n + m)$ due to rolling hash
  Worst case: $O(n * m)$ when many hash collisions occur and full comparisons are needed
- **Space Complexity:**
  $O(1)$ since only constant extra variables are used

20. Z Function

Given a **text string** and a **pattern string**, you need to find **all starting indices** where the pattern occurs in the text using the **Z-function algorithm**.
All indices must be returned using **0-based indexing**.

Example
text = `"xyzabxyzabxyz"`
pattern = `"xyz"`

The pattern `"xyz"` appears starting at indices `0`, `5`, and `10`.
Output = `[0, 5, 10]`

Approach

Using Z-Function Algorithm

Algorithm

1. The Z-function helps find prefix matches efficiently in linear time.
2. Construct a new string by joining:
    - `pattern + "$" + text`
      ($ is a separator that does not appear in pattern or text)
3. Compute the Z-array for this combined string.
    - `Z[i]` tells how many characters starting at index `i` match the prefix of the string.
4. Traverse the Z-array:
    - If `Z[i] == pattern.length()`, it means the pattern matches the text starting at:
      `i - pattern.length() - 1`
5. Collect all such indices and return them.

Why this works

- The prefix of the combined string is the pattern.
- A full prefix match at any position directly indicates a pattern occurrence.
- Previously computed matches are reused using a sliding window, avoiding repeated comparisons.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    vector<int> computeZArray(const string& s) {
        int n = s.length();
        vector<int> z(n, 0);
        int left = 0, right = 0;

        for (int i = 1; i < n; i++) {
            if (i <= right)
                z[i] = min(right - i + 1, z[i - left]);

            while (i + z[i] < n && s[z[i]] == s[i + z[i]])
                z[i]++;

            if (i + z[i] - 1 > right) {
                left = i;
                right = i + z[i] - 1;
            }
        }
        return z;
    }

    vector<int> zFunctionSearch(const string& text, const string& pattern) {
        string combined = pattern + "$" + text;
        vector<int> z = computeZArray(combined);
        vector<int> result;

        for (int i = pattern.length() + 1; i < combined.length(); i++) {
            if (z[i] == pattern.length()) {
                result.push_back(i - pattern.length() - 1);
            }
        }
        return result;
    }
};

int main() {
    string text = "xyzabxyzabxyz";
    string pattern = "xyz";

    Solution sol;
    vector<int> indices = sol.zFunctionSearch(text, pattern);
```

```
    for (int idx : indices) {
        cout << idx << " ";
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n + m)
  The Z-array is computed in linear time using a sliding window technique.
- **Space Complexity:** O(n + m)
  Extra space is used for the combined string and the Z-array.

21. KMP Algorithm or LPS Array

Given a **text string** and a **pattern string**, you need to find **all starting indices** where the pattern occurs in the text using **0-based indexing**.
The indices must be returned in **ascending order**.
If the pattern does not occur in the text, return an **empty list**.

The solution must be implemented using the **Knuth-Morris-Pratt (KMP) algorithm**, which is an efficient pattern matching technique.

Example
text = `"abracadabra"`
pattern = `"abra"`

The pattern `"abra"` occurs at index `0` and `7`.
Output = `[0, 7]`

Approach

KMP Algorithm using LPS Array

Algorithm

1. When matching a pattern in a text using a simple approach, repeated comparisons happen after mismatches.
2. KMP avoids this by using the **LPS (Longest Prefix which is also Suffix)** array.
3. The LPS array tells how many characters can be skipped in the pattern after a mismatch.
4. First, build the LPS array for the pattern:
    - `lps[i]` stores the length of the longest prefix of the pattern that is also a suffix ending at index `i`.
5. Start comparing characters of the text and pattern:
    - If characters match, move both pointers forward.
    - If a mismatch occurs:
        - Use the LPS array to shift the pattern pointer without moving the text pointer backward.
6. When the entire pattern is matched:
    - Store the starting index.
    - Continue searching using the LPS value.
7. Continue until the text is fully processed.

Example Walkthrough

text = `"abcabcabc"`

pattern = `"abc"`

Matches found at:

- index 0
- index 3
- index 6

text = "ababc"
pattern = "abc"

LPS array of pattern =

0	1	2
0	0	0

text =

0	1	2	3	4
a	b	a	b	c

i

LPS array of pattern =

0	1	2
a	b	c

j

i = 0, **j =** 0 → "a" = "a", increment i and j

i = 1, **j =** 1 → "b" = "b", increment i and j

i = 2, **j =** 2 → "a" ≠ "c", **j = LPS[j-1] =** 0

i = 2, **j =** 0 → "a" = "a", increment i and j

i = 3, **j =** 1 → "b" = "b", increment i and j

i = 4, **j =** 2 → "c" = "c", increment i and j

Hence, **j =** 3, a complete pattern found at index, **i-3 = 5-3** = 2

text = "ababc"
pattern = "abc"

LPS array of pattern =

0	1	2
0	0	0

text =

0	1	2	3	4
a	b	a	b	c

i

LPS array of pattern =

0	1	2
a	b	c

j

i = 0, **j =** 0 → "a" = "a", increment i and j

i = 1, **j =** 1 → "b" = "b", increment i and j

i = 2, **j =** 2 → "a" ≠ "c", **j = LPS[j-1] =** 0

i = 2, **j =** 0 → "a" = "a", increment i and j

i = 3, **j =** 1 → "b" = "b", increment i and j

i = 4, **j =** 2 → "c" = "c", increment i and j

Hence, **j =** 3, a complete pattern found at index, **i-3 = 5-3** = 2

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> computeLPS(string pattern) {
        vector<int> lps(pattern.length(), 0);
        int len = 0;
        int i = 1;

        while (i < pattern.length()) {
            if (pattern[i] == pattern[len]) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len != 0) {
                    len = lps[len - 1];
                } else {
                    lps[i] = 0;
                    i++;
                }
            }
        }
        return lps;
    }

    vector<int> KMP(string text, string pattern) {
        vector<int> lps = computeLPS(pattern);
        vector<int> result;

        int i = 0;
        int j = 0;

        while (i < text.length()) {
            if (text[i] == pattern[j]) {
                i++;
                j++;
            }

            if (j == pattern.length()) {
```

```
            result.push_back(i - j);
            j = lps[j - 1];
        }
        else if (i < text.length() && text[i] != pattern[j]) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return result;
  }
};

int main() {
  string text = "abracadabra";
  string pattern = "abra";

  Solution sol;
  vector<int> indices = sol.KMP(text, pattern);

  for (int idx : indices) {
    cout << idx << " ";
  }
  return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n + m)
  LPS array construction takes O(m) and pattern matching takes O(n).
- **Space Complexity:** O(m)
  Extra space is used only for the LPS array.

22. Shortest Palindromic Substring

Given a string, you need to find the **shortest palindromic substring** present in it.
If there are multiple shortest palindromic substrings, return the **lexicographically smallest** one.

A palindrome is a string that reads the same forward and backward.

Example
Input: `"zyzz"`
Palindromic substrings are `"z"`, `"y"`, `"zz"`, `"yzz y"`
Shortest length is 1, lexicographically smallest is `"y"`

Input: `"abab"`
Palindromic substrings of length 1 are `"a"` and `"b"`
Lexicographically smallest is `"a"`

Approach 1

Naive Approach (Expand Around Center)

Algorithm

1. Every palindrome has a center.
2. For each index `i` in the string:
    - Treat `i` as the center for **odd length** palindromes.
    - Treat `i-1` and `i` as centers for **even length** palindromes.
3. Expand left and right while characters are equal to form palindromes.
4. Store all found palindromic substrings in a vector.
5. Ignore empty substrings.
6. From all stored palindromes, select the **shortest one**.
7. If multiple have the same length, choose the **lexicographically smallest**.

Example Dry Run
s = `"abab"`
Centers generate palindromes: `"a"`, `"b"`, `"a"`, `"b"`, `"aba"`, `"bab"`
Shortest length = 1 → `"a"` is lexicographically smallest.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
string ShortestPalindrome(string s) {
    int n = s.length();
    vector<string> v;

    for (int i = 0; i < n; i++) {
        int l = i, r = i;
        string ans1 = "", ans2 = "";

        // odd length palindrome
        while (l >= 0 && r < n && s[l] == s[r]) {
            ans1 += s[l];
            l--;
            r++;
        }

        l = i - 1;
        r = i;

        // even length palindrome
        while (l >= 0 && r < n && s[l] == s[r]) {
            ans2 += s[l];
            l--;
            r++;
        }

        v.push_back(ans1);
        v.push_back(ans2);
    }

    string ans = "";
    for (int i = 0; i < v.size(); i++) {
        if (v[i] != "") {
            if (ans == "" || v[i].length() < ans.length() ||
               (v[i].length() == ans.length() && v[i] < ans)) {
                ans = v[i];
            }
        }
    }
    return ans;
}

int main() {
    string s = "geeksforgeeks";
    cout << ShortestPalindrome(s);
```

```
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N²)
  Each character is treated as a center and expanded in both directions.
- **Space Complexity:** O(N²)
  All palindromic substrings can be stored in the vector in the worst case.

Approach 2

Efficient Approach (Observation-Based)

Algorithm

1. A **single character** is always a palindrome.
2. The shortest possible palindromic substring has length 1.
3. So, the answer will always be the **lexicographically smallest character** in the string.
4. Traverse the string and track the minimum character.
5. Return that character.

Reasoning
Any longer palindrome will have length ≥ 2, which can never be shorter than a single character. So checking individual characters is sufficient.

Code

```
#include <bits/stdc++.h>
using namespace std;

char ShortestPalindrome(string s) {
    char ans = s[0];
    for (int i = 1; i < s.length(); i++) {
        ans = min(ans, s[i]);
    }
    return ans;
}

int main() {
    string s = "geeksforgeeks";
    cout << ShortestPalindrome(s);
    return 0;
```

}

Complexity Analysis

- **Time Complexity:** O(N)
  Single traversal of the string.
- **Space Complexity:** O(1)
  Only one variable is used.

23. Longest Happy Prefix

Given a string `s`, you need to find the **longest happy prefix** of the string.
A happy prefix is a string that is **both a proper prefix and a proper suffix**.

A **proper prefix / suffix** means it must **not be equal to the entire string**.

If no such prefix exists, return an **empty string**.

Example
Input: `s = "ababab"`
Prefixes: `"a","ab","aba","abab","ababa"`
Suffixes: `"b","ab","bab","abab","babab"`
Longest common proper prefix and suffix is `"abab"`

Output: `"abab"`

Approach 1

Brute Force Approach

Algorithm

1. The maximum possible length of a proper prefix or suffix is `n-1`.
2. Start checking from length `n-1` down to 1.

3. For each length:
    - Compare the prefix `s[0 … len-1]`
    - With the suffix `s[n-len … n-1]`
4. If both are equal, return that prefix immediately because we are checking from longest to shortest.
5. If no match is found, return an empty string.

This approach works but performs repeated comparisons.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    string longestPrefix(string s) {
        int n = s.size();

        for (int len = n - 1; len > 0; len--) {
            if (s.substr(0, len) == s.substr(n - len, len)) {
                return s.substr(0, len);
            }
        }
        return "";
    }
};

int main() {
    Solution sol;
    string s = "levellevel";
    cout << sol.longestPrefix(s);
    return 0;
}
```

Complexity Analysis

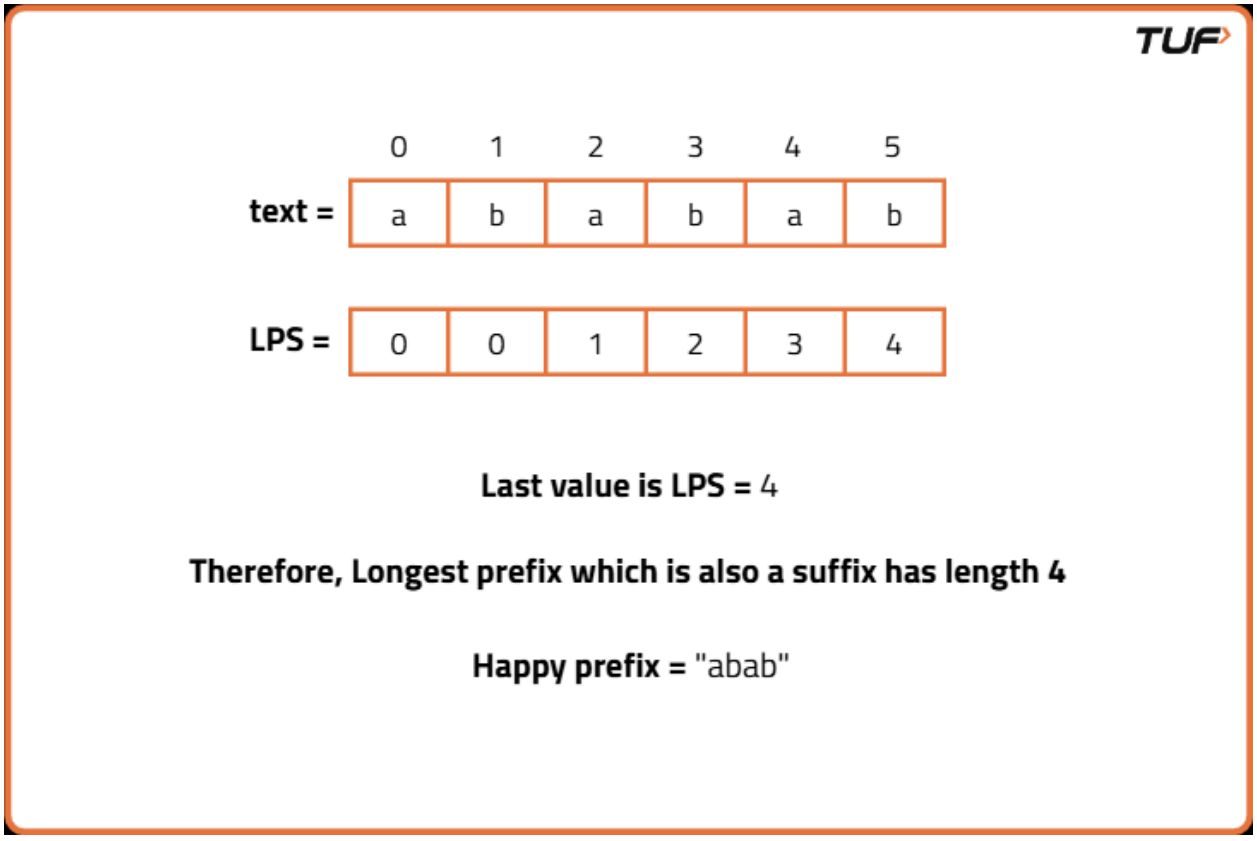
- **Time Complexity:** O(n²)
  For each possible length, substring comparison can take up to n time.
- **Space Complexity:** O(n)
  Temporary substrings are created during comparison.

Approach 2

Optimal Approach (Using KMP / LPS Array)

Algorithm

1. Build the **LPS (Longest Prefix Suffix)** array for the string.
2. `lps[i]` stores the length of the longest prefix that is also a suffix ending at index `i`.
3. Start from the second character and maintain a pointer `len` for the current prefix length.
4. If characters match, increase `len` and store it in `lps`.
5. If a mismatch occurs:
    - Move `len` back using the LPS array instead of restarting.
6. After processing the whole string:
    - The last value of the LPS array gives the length of the longest happy prefix.
7. Return the substring from index `0` to that length.



Example Dry Run

`s = "aaaa"`

LPS array = `[0,1,2,3]`

Last LPS value = 3

Answer = `"aaa"`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    string longestPrefix(string s) {
        vector<int> lps(s.length(), 0);
        int len = 0;

        for (int i = 1; i < s.length(); i++) {
            if (s[i] == s[len]) {
                len++;
                lps[i] = len;
            } else if (len != 0) {
                len = lps[len - 1];
                i--;
            }
        }

        return s.substr(0, lps[s.length() - 1]);
    }
};

int main() {
    Solution sol;
    string s = "levellevel";
    cout << sol.longestPrefix(s);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n)
  The string is processed once while building the LPS array.
- **Space Complexity:** O(n)
  Extra space is used for the LPS array.

24. Count Palindromic Subsequence in Given String

Given a string s, you need to **count the total number of distinct palindromic subsequences** present in it.
A **palindromic subsequence** is obtained by deleting zero or more characters (without changing order) such that the resulting string reads the same forward and backward.

Only **distinct** palindromic subsequences are counted.

Example
Input: `s = "bccb"`
Palindromic subsequences are: `"b", "c", "bb", "cc", "bcb", "bccb"`
Output: 6

Input: `s = "abcd"`
Only single characters are palindromes: `"a","b","c","d"`
Output: 4

Approach 1

Brute Force Approach

Algorithm

1. Generate **all possible subsequences** of the string using recursion.
2. For each subsequence:
    - Ignore empty subsequences.
    - Check if it is a palindrome.
3. Count every palindromic subsequence found.
4. Return the total count.

This approach directly follows the definition but is very slow.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
```

```
    bool isPalindrome(string s) {
        int l = 0, r = s.size() - 1;
        while (l < r) {
            if (s[l++] != s[r--]) return false;
        }
        return true;
    }

    int count(string s, string curr, int index) {
        if (index == s.length()) {
            if (!curr.empty() && isPalindrome(curr)) return 1;
            return 0;
        }

        int include = count(s, curr + s[index], index + 1);
        int exclude = count(s, curr, index + 1);

        return include + exclude;
    }

    int countPalindromicSubsequences(string s) {
        return count(s, "", 0);
    }
};

int main() {
    Solution sol;
    string s = "abcb";
    cout << sol.countPalindromicSubsequences(s);
    return 0;
}
```

Complexity Analysis

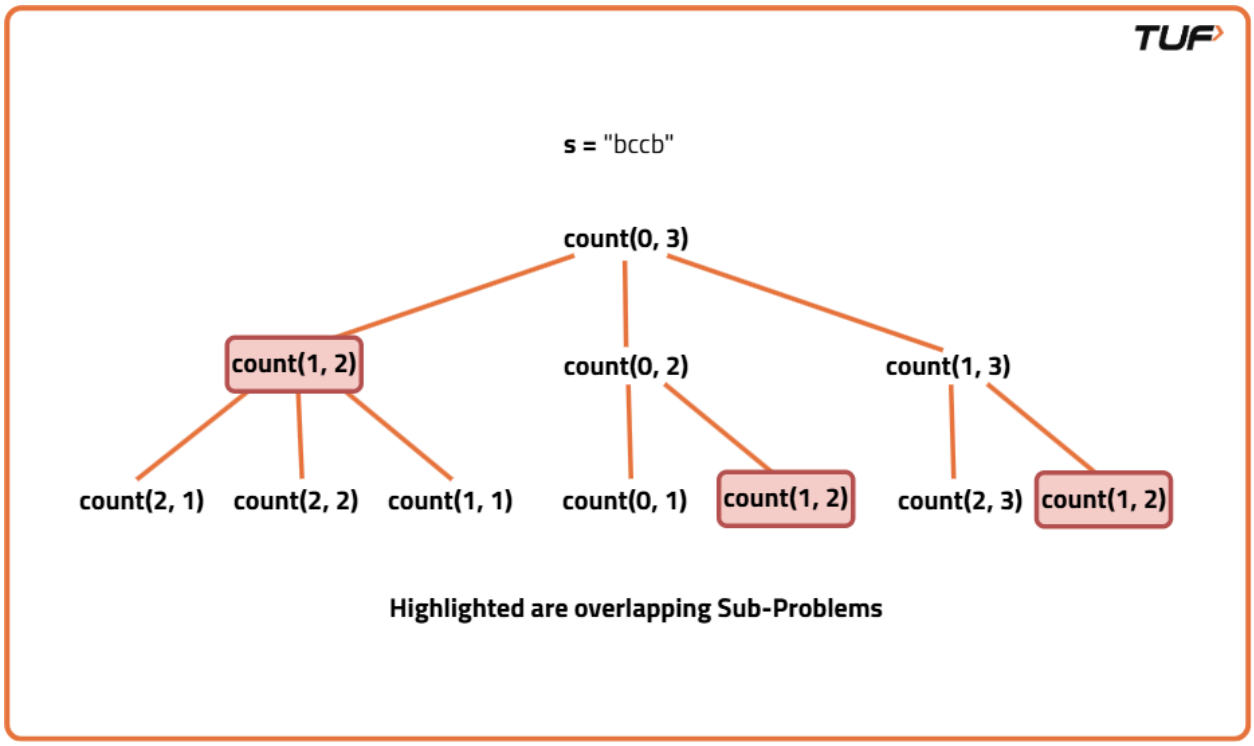
- **Time Complexity:** $O(2^n \cdot n)$
  All subsequences are generated and each is checked for palindrome.
- **Space Complexity:** $O(n)$
  Recursion stack and temporary strings.

Approach 2

Optimal Approach – Memoization

Algorithm

1. Use recursion on substring ranges `[i…j]`.
2. Base cases:
    - If `i > j`, return `0`.
    - If `i == j`, return 1 (single character).
3. If result already computed, return it.
4. If `s[i] == s[j]`:
    - Count subsequences excluding first
    - Count subsequences excluding last
    - Add 1 for the new palindrome formed by `s[i]` and `s[j]`
5. If `s[i] != s[j]`:
    - Add counts from excluding first and last
    - Subtract overlapping part to avoid double count
6. Store result in DP table.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countPalindromicSubsequences(string s) {
```

```
        int n = s.size();
        vector<vector<int>> dp(n, vector<int>(n, -1));
        return count(s, 0, n - 1, dp);
    }

private:
    int count(string &s, int i, int j, vector<vector<int>> &dp) {
        if (i > j) return 0;
        if (i == j) return 1;
        if (dp[i][j] != -1) return dp[i][j];

        if (s[i] == s[j])
            dp[i][j] = count(s, i + 1, j, dp) +
                       count(s, i, j - 1, dp) + 1;
        else
            dp[i][j] = count(s, i + 1, j, dp) +
                       count(s, i, j - 1, dp) -
                       count(s, i + 1, j - 1, dp);

        return dp[i][j];
    }
};

int main() {
    Solution sol;
    string s = "aab";
    cout << sol.countPalindromicSubsequences(s);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n²)
  Each substring `[i…j]` is solved once.
- **Space Complexity:** O(n² + n)
  DP table + recursion stack.

Approach 3

Optimal Approach – Tabulation

Algorithm

1. Create a 2D DP table `dp[i][j]` to store number of palindromic subsequences in substring `[i…j]`.
2. Initialize:
    - `dp[i][i] = 1` (single characters).
3. Consider substrings of increasing length:
    - If `s[i] == s[j]`:
      `dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1`
    - Else:
      `dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1]`
4. Final answer is `dp[0][n-1]`.

This avoids recursion and is fully iterative.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   int countPalindromicSubsequences(string s) {
      int n = s.size();
      vector<vector<int>> dp(n, vector<int>(n, 0));

      for (int i = 0; i < n; i++) {
         dp[i][i] = 1;
      }

      for (int len = 2; len <= n; len++) {
         for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;

            if (s[i] == s[j])
               dp[i][j] = dp[i + 1][j] + dp[i][j - 1] + 1;
            else
               dp[i][j] = dp[i + 1][j] + dp[i][j - 1] -
                        dp[i + 1][j - 1];
         }
      }
      return dp[0][n - 1];
   }
};

int main() {
```

```
    Solution sol;
    string s = "aab";
    cout << sol.countPalindromicSubsequences(s);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n²)
  All substrings are processed once.
- **Space Complexity:** O(n²)
  DP table stores results for all substrings.